

# 2026 年操作系统原理与设计

## 期末回忆版

### 一、填空题

1. 进程是\_\_\_\_\_，其地址空间通常包含\_\_\_\_\_、\_\_\_\_\_、\_\_\_\_\_、\_\_\_\_\_等区域。
2. 常用的三类线程库是\_\_\_\_\_、\_\_\_\_\_、\_\_\_\_\_。
3. 进程优先级翻转可能导致\_\_\_\_\_问题，常用解决方法是\_\_\_\_\_。

### 二、名词解释

1. 空闲让进/进步 (progress)。
2. 多级反馈队列调度 (multilevel feedback queue, MLFQ)。
3. 资源分配图 (resource-allocation graph)。
4. 磁盘调度中的 SCAN 算法。
5. I/O 控制方式。

### 三、简答题

1. 什么是系统调用？它在用户程序和操作系统内核之间起什么作用？系统调用的参数和返回值通常分别通过哪些方式传递？
2. 从多核处理器与就绪队列的对应关系来看，操作系统维护就绪队列通常有两种组织方式。这两种方式分别是什么？请说明各自的优点和缺点。
3. 某系统采用多级页表。给定一个虚拟地址 VA 以及该地址对应的页表项 PTE。题目给出了虚拟地址中各级页表索引和页内偏移的位数，并给出了相应计算公式；PTE 中包含访问权限位和状态位。  
已知该 PTE 表示：该页为共享页、用户页、可读、不可写，并被标记为 copy-on-write (COW) 页。

- (1) 根据题目给出的位宽和公式，计算各级页表索引以及页内偏移。
  - (2) 现在对该虚拟地址发起一次写操作。该写操作能否直接完成？若不能，请说明硬件和操作系统会依次发生什么。
4. FAT16 文件系统相关计算。已知簇号从 2 开始编号，首数据扇区号为 51，每簇包含 4 个扇区，每个扇区大小为 512 字节。某文件的簇链为

$5 \rightarrow 9 \rightarrow 13 \rightarrow \text{EOF}.$

- (1) 计算簇 9 和簇 13 对应的起始数据扇区号。
- (2) 不考虑读取 FAT 表本身的开销，也不考虑 cache。若要读取该文件偏移 3000 字节到 5200 字节范围内的数据，说明该范围跨越哪些簇，并计算至少需要读取多少个数据扇区。

## 四、综合题

1. 关于抖动 (thrashing)，回答下列问题：
  - (1) 什么是抖动？
  - (2) 操作系统可以如何检测抖动？检测到抖动后可以如何处理？
  - (3) 操作系统可以如何预防抖动？
2. 下面是一段异步服务中的伪代码。系统中有一个全局请求表锁 `svc_sem`，以及请求对象 `r` 自身的锁 `r_sem`。函数 `find_req_by_reqid()` 用于在请求表中查找与给定 `reqid` 匹配的请求。若找到目标请求，它返回请求对象指针，并保持该请求对象锁处于持有状态；调用者使用完请求对象后应释放该锁。

```
semaphore r_sem = 1;          // lock for request r
semaphore svc_sem = 1;        // lock for request table
request r;

request *find_req_by_reqid(pid_t pid, int reqid) {
    request *ret = NULL;

    down(&svc_sem);
    if (request_table_contains(pid, reqid) && r.valid) {
        down(&r_sem);
        ret = &r;
    }
    up(&svc_sem);

    return ret;                // if ret != NULL, r_sem is still held
}
```

```
}

int thread_A(pid_t pid, int reqid) {
    request *req = find_req_by_reqid(pid, reqid);
    if (req == NULL)
        return -1;

    if (req->state == DONE) {
        up(&r_sem);
        return 0;
    }

    up(&r_sem);
    return 1;
}

void thread_B(pid_t pid, int reqid) {
    down(&svc_sem);
    if (request_table_contains(pid, reqid) && r.valid) {
        down(&r_sem);
        process_request(&r);
        r.state = DONE;
        up(&r_sem);
    }
    up(&svc_sem);
}
```

回答下列问题：

- (1) 什么是信号量？信号量在并发程序中有什么作用？
- (2) 死锁发生的四个必要条件是什么？
- (3) 根据上述伪代码判断是否可能发生死锁。若不会发生，请说明理由；若会发生，请写出一个可能的死锁执行序列，并给出一种解除或避免该死锁的方法。